

docs_chain

Docs Chain

Tagline: No tracked document can fall out of sync — content-hashed, bidirectional, atomic.

Summary: Docs Chain is a universal, Go-implemented bidirectional document-and-database dependency-propagation engine. When any member of a registered chain changes — Markdown source, an HTML/PDF/DOCX export, or a SQLite database — it detects the change by content hash and propagates it through every connected member atomically.

Short description (~40 words): A Go engine that keeps documents and databases in sync. Using Salsa-style content-hashed incremental recomputation over a DAG (Kahn topological ordering, early cutoff, bidirectional sync edges, atomic-rename + SQLite-transaction commits), it regenerates exports whenever any linked artifact changes.

Long description (150-250 words): Docs Chain is the mechanical successor to ad-hoc documentation-sync scripts. It models a project's documents and databases as members of a chain and, when any member changes, propagates that change through every connected member in every declared direction — regenerating and re-exporting atomically so no tracked artifact can drift out of sync. Change detection is by **content hash, not mtime**, which avoids spurious rebuilds and catches real edits reliably. Its formal model is a single line: Salsa-style content-hashed incremental recomputation over a DAG, with Kahn topological ordering, early cutoff (skip unchanged subtrees), declared-authority bidirectional sync edges, and atomic-rename plus SQLite-transaction commits. It ships as a `vasic-digital` submodule and is consumed as a core part of the `HelixConstitution` submodule, so any project that adopts the constitution gets Docs Chain out of the box and registers its own chains via per-context YAML. The implementation is honest about status (per constitution §11.4.6): Phases 1-4 (core DAG + hashing, node adapters/transforms, propagation orchestrator with atomicity, config-driven multi-context CLI with `sync/verify/doctor/graph/watch`) are implemented and tested; a Phase-4b adds generic

bidirectional md-to-sqlite/sqlite-to-md builtins (pure-Go, row-level drift, byte-stable round-trip) and a `colorize-html` builtin; Phase 5 comprehensive real-binary e2e is implemented and GREEN. Phases 6–7 (constitution distribution, ATMOSphere wiring) remain PLANNED and operator-gated. Herald is the first real downstream consumer, syncing a 66-document multi-format corpus that verifies clean.

Why we built it: Documentation, exports, and databases drift apart the moment they're maintained by hand or by fragile scripts. Docs Chain makes synchronization mechanical, content-hash-accurate, and atomic, so a change anywhere in a chain correctly and safely updates everything downstream (and upstream).

Why it's a game-changer: It brings incremental-build rigor (Salsa/DAG/early-cutoff) to documentation and databases, with true bidirectional sync and atomic commits — eliminating an entire class of “docs are out of date” and “export doesn't match source” defects.

What's innovative: - Content-hash (not mtime) incremental recompute over a DAG with early cutoff. - Bidirectional, declared-authority sync edges (docs ↔ exports ↔ SQLite). - Atomic-rename + SQLite-transaction commit for crash-safe propagation. - Pure-Go md-to-sqlite/sqlite-to-md round-trip with row-level drift detection.

Biggest technical challenges + how solved: - *Spurious rebuilds*: solved by content-hash detection instead of timestamps. - *Partial/corrupt updates*: solved with atomic-rename and SQLite transactions. - *Correct multi-member ordering*: solved with Kahn topological ordering + early cutoff. - *Honest capability reporting*: solved by marking each phase IMPLEMENTED vs PLANNED per §11.4.6.

Tech stack (why + how): - **Go** — entire engine (internal/hash, graph, adapter, orchestrator, config, state, runner, cmd/docs_chain). - **DAG + Kahn topological sort** — dependency ordering with early cutoff. - **SQLite (pure-Go modernc)** — DB members and transactional commits. - **fsnotify** — watch daemon for live propagation. - **YAML config** — per-context chain registration. - **exec: transforms** — pluggable Markdown→HTML/PDF/DOCX generation.

Public links: - GitHub (vasic-digital): https://github.com/vasic-digital/docs_chain (public). - Distributed via the HelixConstitution submodule to consuming projects. - Note: Phases 6–7 are PLANNED/operator-gated (do not present as shipped).

Suggested diagrams/illustrations (OpenDesign): 1. DAG of chain members (Markdown ↔ HTML/PDF/DOCX ↔ SQLite) with a change propagating. 2. Content-hash vs mtime comparison (why hashing wins). 3. Atomic-commit sequence (temp write → rename / SQLite txn). 4. Phase status board (implemented vs planned).

Site relevance: vasic.digital (engineering-depth / developer-tooling). A strong “correctness engineering” showcase.

Priority tier: vasic-util-secondary

Source provenance: gh repo view vasic-digital/docs_chain README (model, phase status table, builtins, consumers, §11.4.6 honesty note); _analysis/github-vasic-digital.md (description, size).